

```

# =====
# MODULE 1: INSTALLATION, IMPORTS AND CONFIGURATION
# SPORTS IMAGE CLASSIFICATION
# =====

# Install required libraries
!pip -q install kagglehub seaborn scikit-learn pandas pillow opencv-python

# -----
# IMPORT LIBRARIES
# -----

import os
import re
import time
import hashlib
import random
import warnings

from pathlib import Path

import numpy as np
import pandas as pd

from PIL import Image

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    classification_report,
    confusion_matrix,
    roc_auc_score
)

# -----
# RANDOM SEED
# -----
SEED = 42

```

```

SEED = 42

random.seed(SEED)
np.random.seed(SEED)
tf.random.set_seed(SEED)

# -----
# CONFIGURATION
# -----

IMG_SIZE = (224, 224)

BATCH_SIZE = 8

# Use 1 epoch for testing.
# Change to 3-10 for final training.
EPOCHS = 1

AUTOTUNE = tf.data.AUTOTUNE

# Dataset and output directories
DATASET_DIR = Path("/content/sports_dataset")

OUTPUT_DIR = Path("/content/sports_outputs")

OUTPUT_DIR.mkdir(
    parents=True,
    exist_ok=True
)

# -----
# GPU CHECK
# -----

print("TensorFlow version:", tf.__version__)

gpus = tf.config.list_physical_devices("GPU")

if gpus:
    print("GPU available:", gpus)
else:
    print("WARNING: GPU not available. Enable T4 GPU in Colab.")

# -----
# BASIC INFORMATION
# -----

print("Image size:", IMG_SIZE)
print("Batch size:", BATCH_SIZE)
print("Epochs:", EPOCHS)
print("Output directory:", OUTPUT_DIR)

```

```
print("\nMODULE 1 COMPLETED")
```

```
TensorFlow version: 2.20.0
GPU available: [PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU')]
Image size: (224, 224)
Batch size: 8
Epochs: 1
Output directory: /content/sports_outputs
```

```
MODULE 1 COMPLETED
```

```
# =====
# MODULE 2: DATASET DOWNLOAD, CLEANING AND DATA SPLITTING
# =====

import kagglehub

# -----
# DOWNLOAD DATASET
# -----

print("Downloading dataset...")

downloaded_path = kagglehub.dataset_download(
    "gpiosenka/sports-classification"
)

DATASET_DIR = Path(downloaded_path)

print("Dataset directory:", DATASET_DIR)

# -----
# FIND ALL IMAGE FILES
# -----

IMAGE_EXTENSIONS = {
    ".jpg",
    ".jpeg",
    ".png",
    ".bmp",
    ".webp"
}

records = []

for image_path in DATASET_DIR.rglob("*"):
    if (
        image_path.is_file()
        and image_path.suffix.lower() in IMAGE_EXTENSIONS
    ):

```

```
# Parent folder is treated as the class label
label = image_path.parent.name

records.append(
    (
        str(image_path),
        label
    )
)

# Create DataFrame
df = pd.DataFrame(
    records,
    columns=[
        "path",
        "label"
    ]
)

# Remove duplicate file paths
df = df.drop_duplicates(
    subset=["path"]
).reset_index(drop=True)

print("Total raw images:", len(df))

print(
    "Total classes:",
    df["label"].nunique()
)

print("\nInitial class distribution:")
print(df["label"].value_counts())

# -----
# CHECK IMAGE VALIDITY
# -----

def valid_image(image_path):

    try:

        with Image.open(image_path) as image:

            image.verify()

        return True

    except Exception:
```

```
        return False

print("\nChecking invalid images...")

df["valid"] = df["path"].apply(valid_image)

invalid_count = (~df["valid"]).sum()

print("Invalid images:", invalid_count)

# Keep only valid images
df = df[df["valid"]].drop(
    columns=["valid"]
).reset_index(drop=True)

print(
    "Images after invalid-image removal:",
    len(df)
)

# -----
# REMOVE EXACT DUPLICATES
# USING SHA-256 HASH
# -----

def calculate_sha256(file_path):

    hash_object = hashlib.sha256()

    with open(
        file_path,
        "rb"
    ) as file:

        while True:

            data = file.read(1024 * 1024)

            if not data:
                break

            hash_object.update(data)

    return hash_object.hexdigest()

print("\nCalculating image hashes...")

df["sha256"] = df["path"].apply(
    calculate_sha256
```

```
)

duplicate_count = df.duplicated(
    subset=["sha256"]
).sum()

print(
    "Exact duplicate images:",
    duplicate_count
)

# Remove duplicate images
df = df.drop_duplicates(
    subset=["sha256"]
).drop(
    columns=["sha256"]
).reset_index(drop=True)

print(
    "Images after duplicate removal:",
    len(df)
)

# -----
# CLASS LABEL ENCODING
# -----

class_names = sorted(
    df["label"].unique()
)

class_to_id = {
    class_name: index
    for index, class_name in enumerate(class_names)
}

df["class_id"] = df["label"].map(
    class_to_id
)

NUM_CLASSES = len(class_names)

print("\nClass names:")

for index, class_name in enumerate(class_names):

    print(index, ":", class_name)

print(
    "\nTotal number of classes:",
    NUM_CLASSES
)
```

```
)

# -----
# CLASS DISTRIBUTION
# -----

class_distribution = (
    df["label"]
    .value_counts()
    .sort_index()
)

plt.figure(figsize=(18, 6))

class_distribution.plot(
    kind="bar"
)

plt.title("Class Distribution")

plt.xlabel("Sports Classes")

plt.ylabel("Number of Images")

plt.xticks(
    rotation=90
)

plt.tight_layout()

plt.show()

# -----
# SAVE CLEANED DATASET INDEX
# -----

cleaned_index_path = (
    OUTPUT_DIR / "cleaned_dataset_index.csv"
)

df.to_csv(
    cleaned_index_path,
    index=False
)

print(
    "\nCleaned dataset index saved at:",
    cleaned_index_path
)

# -----
```

```
# STRATIFIED 70/15/15 SPLIT
# -----

# First split:
# 70% training and 30% temporary data

train_df, temporary_df = train_test_split(

    df,

    test_size=0.30,

    stratify=df["class_id"],

    random_state=SEED
)

# Second split:
# Temporary data divided equally:
# 15% validation and 15% testing

validation_df, test_df = train_test_split(

    temporary_df,

    test_size=0.50,

    stratify=temporary_df["class_id"],

    random_state=SEED
)

# -----
# RESET DATAFRAME INDEXES
# -----

train_df = train_df.reset_index(
    drop=True
)

validation_df = validation_df.reset_index(
    drop=True
)

test_df = test_df.reset_index(
    drop=True
)

# -----
# DISPLAY SPLIT INFORMATION
# -----
```

```
print("\nDATASET SPLIT INFORMATION")

print(
    "Training images:",
    len(train_df)
)

print(
    "Validation images:",
    len(validation_df)
)

print(
    "Testing images:",
    len(test_df)
)

total_images = (
    len(train_df)
    + len(validation_df)
    + len(test_df)
)

print(
    "Total images:",
    total_images
)

print(
    "Training percentage:",
    round(len(train_df) / total_images * 100, 2),
    "%"
)

print(
    "Validation percentage:",
    round(len(validation_df) / total_images * 100, 2),
    "%"
)

print(
    "Testing percentage:",
    round(len(test_df) / total_images * 100, 2),
    "%"
)

# -----
# SAVE DATASET SPLITS
# -----
```

```
train_df.to_csv(
    OUTPUT_DIR / "train_split.csv",
    index=False
)

validation_df.to_csv(
    OUTPUT_DIR / "validation_split.csv",
    index=False
)

test_df.to_csv(
    OUTPUT_DIR / "test_split.csv",
    index=False
)

# -----
# VERIFY DATA LEAKAGE
# -----

train_paths = set(train_df["path"])

validation_paths = set(
    validation_df["path"]
)

test_paths = set(
    test_df["path"]
)

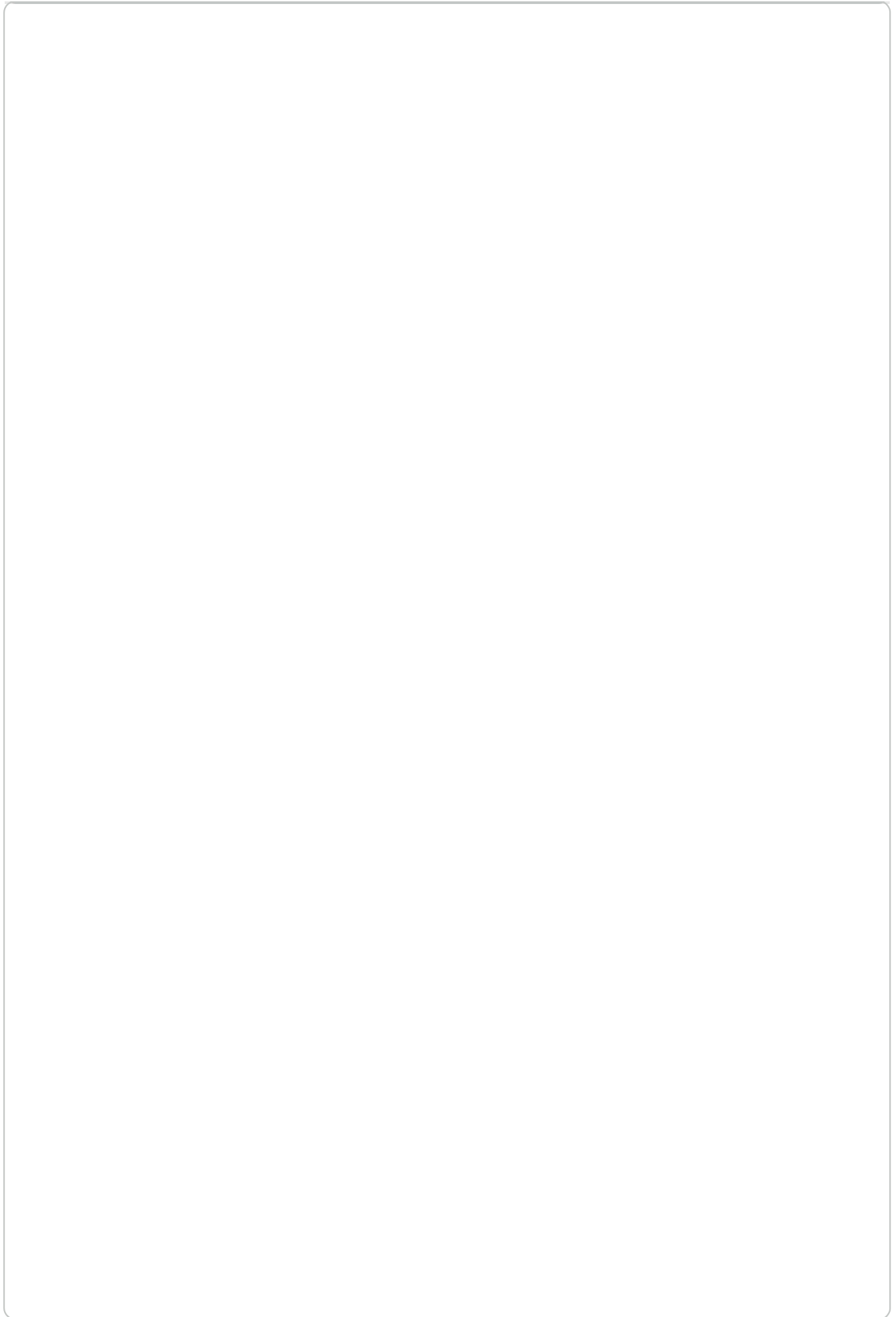
print("\nDATA LEAKAGE CHECK")

print(
    "Train and validation overlap:",
    len(train_paths.intersection(validation_paths))
)

print(
    "Train and test overlap:",
    len(train_paths.intersection(test_paths))
)

print(
    "Validation and test overlap:",
    len(validation_paths.intersection(test_paths))
)

print("\nMODULE 2 COMPLETED")
```



```
Downloading dataset...
Downloading to /root/.cache/kagglehub/datasets/gpiosenka/sports-classification
100%|██████████| 424M/424M [00:03<00:00, 122MB/s]Extracting files...

Dataset directory: /root/.cache/kagglehub/datasets/gpiosenka/sports-classification
Total raw images: 14492
Total classes: 100

Initial class distribution:
label
football                201
formula 1 racing        200
nascar racing           199
baseball                184
hockey                  182
...
horseshoe pitching      112
arm wrestling            109
wingsuit flying         108
ultimate                107
sky surfing              69
Name: count, Length: 100, dtype: int64

Checking invalid images...
Invalid images: 0
Images after invalid-image removal: 14492

Calculating image hashes...
Exact duplicate images: 0
Images after duplicate removal: 14492

Class names:
0 : air hockey
1 : ampute football
2 : archery
3 : arm wrestling
4 : axe throwing
5 : balance beam
6 : barell racing
7 : baseball
8 : basketball
9 : baton twirling
10 : bike polo
11 : billiards
12 : bmx
13 : bobsled
14 : bowling
15 : boxing
16 : bull riding
17 : bungee jumping
18 : canoe slamon
19 : cheerleading
20 : chuckwagon racing
21 : cricket
22 : croquet
23 : curling
```

```
24 : disc golf
25 : fencing
26 : field hockey
27 : figure skating men
28 : figure skating pairs
29 : figure skating women
30 : fly fishing
31 : football
32 : formula 1 racing
33 : frisbee
34 : gaga
35 : giant slalom
36 : golf
37 : hammer throw
38 : hang gliding
39 : harness racing
40 : high jump
41 : hockey
42 : horse jumping
43 : horse racing
44 : horseshoe pitching
45 : hurdles
46 : hydroplane racing
47 : ice climbing
48 : ice yachting
49 : jai alai
50 : javelin
51 : jousting
52 : judo
53 : lacrosse
54 : log rolling
55 : luge
56 : motorcycle racing
57 : mushing
58 : nascar racing
59 : olympic wrestling
60 : parallel bar
61 : pole climbing
62 : pole dancing
63 : pole vault
64 : polo
65 : pommel horse
66 : rings
67 : rock climbing
68 : roller derby
69 : rollerblade racing
70 : rowing
71 : rugby
72 : sailboat racing
73 : shot put
74 : shuffleboard
75 : sidecar racing
76 : ski jumping
77 : sky surfing
78 : skydiving
79 : snow boarding
80 : snowmobile racing
```

```
# =====
# MODULE 3: IMAGE PREPROCESSING AND AUGMENTATION
# =====

# Data augmentation
data_augmentation = keras.Sequential(
    [
        layers.RandomFlip("horizontal"),
        layers.RandomRotation(0.1),
        layers.RandomZoom(0.15),
        layers.RandomContrast(0.1),
    ],
    name="data_augmentation"
)

def load_image(image_path, label):

    image = tf.io.read_file(image_path)

    image = tf.io.decode_image(
        image,
        channels=3,
        expand_animations=False
    )

    image.set_shape([None, None, 3])

    image = tf.image.resize(
        image,
        IMG_SIZE
    )

    image = tf.cast(
        image,
        tf.float32
    )

    return image, label

def preprocess_train(image_path, label):

    image, label = load_image(
        image_path,
        label
    )

    image = data_augmentation(
        image,
        training=True
    )
```

```
)

return image, label


def preprocess_validation(image_path, label):

    image, label = load_image(
        image_path,
        label
    )

    return image, label


def create_dataset(dataframe, training=False):

    image_paths = dataframe["path"].values

    labels = dataframe["class_id"].values

    dataset = tf.data.Dataset.from_tensor_slices(
        (
            image_paths,
            labels
        )
    )

    if training:

        dataset = dataset.shuffle(
            buffer_size=len(dataframe),
            seed=SEED
        )

        dataset = dataset.map(
            preprocess_train,
            num_parallel_calls=AUTOTUNE
        )

    else:

        dataset = dataset.map(
            preprocess_validation,
            num_parallel_calls=AUTOTUNE
        )

    dataset = dataset.map(
        lambda image, label: (
            image,
            tf.one_hot(
```

```
        label,
        depth=NUM_CLASSES
    )
),
num_parallel_calls=AUTOTUNE
)

dataset = dataset.batch(
    BATCH_SIZE
)

dataset = dataset.prefetch(
    AUTOTUNE
)

return dataset

# Create datasets
train_dataset = create_dataset(
    train_df,
    training=True
)

validation_dataset = create_dataset(
    validation_df,
    training=False
)

test_dataset = create_dataset(
    test_df,
    training=False
)

print("Training batches:", len(train_dataset))
print("Validation batches:", len(validation_dataset))
print("Testing batches:", len(test_dataset))

# Display sample images
images, labels = next(iter(train_dataset))

plt.figure(figsize=(12, 8))

for i in range(min(9, len(images))):

    plt.subplot(3, 3, i + 1)

    plt.imshow(
        tf.cast(images[i], tf.uint8)
    )
```

```

class_id = tf.argmax(
    labels[i]
).numpy()

plt.title(
    class_names[class_id]
)

plt.axis("off")

plt.tight_layout()
plt.show()

print("MODULE 3 COMPLETED")

```

Training batches: 1268
 Validation batches: 272
 Testing batches: 272

luge



curling



arm wrestling



luge



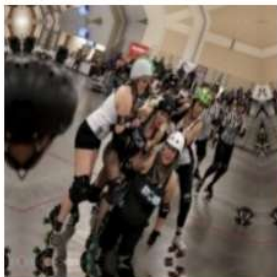
bmx



basketball



roller derby



sidecar racing



MODULE 3 COMPLETED

```

# =====
# MODULE 4: SEVEN DEEP LEARNING ARCHITECTURES
# =====

from tensorflow.keras.applications import (
    ResNet50,

```

```

    EfficientNetB4,
    MobileNetV2,
    Xception,
    DenseNet121
)

from tensorflow.keras.applications import (
    resnet50,
    mobilenet_v2,
    xception,
    densenet
)

# ConvNeXt availability
from tensorflow.keras.applications import ConvNeXtTiny

# -----
# CBAM ATTENTION BLOCK
# -----

def cbam_block(feature_map, reduction_ratio=8):

    channels = feature_map.shape[-1]

    # Channel attention
    average_pool = layers.GlobalAveragePooling2D(
        keepdims=True
    )(feature_map)

    max_pool = layers.GlobalMaxPooling2D(
        keepdims=True
    )(feature_map)

    shared_dense_1 = layers.Dense(
        max(channels // reduction_ratio, 1),
        activation="relu"
    )

    shared_dense_2 = layers.Dense(
        channels,
        activation="sigmoid"
    )

    average_attention = shared_dense_2(
        shared_dense_1(average_pool)
    )

    max_attention = shared_dense_2(
        shared_dense_1(max_pool)
    )

```

```
channel_attention = layers.Add()(
    [
        average_attention,
        max_attention
    ]
)

channel_attention = layers.Activation(
    "sigmoid"
)(channel_attention)

refined_feature = layers.Multiply()(
    [
        feature_map,
        channel_attention
    ]
)

# Spatial attention
average_spatial = layers.Lambda(
    lambda x: tf.reduce_mean(x, axis=-1, keepdims=True)
)(refined_feature)

max_spatial = layers.Lambda(
    lambda x: tf.reduce_max(x, axis=-1, keepdims=True)
)(refined_feature)

spatial_features = layers.Concatenate(
    axis=-1
)(
    [
        average_spatial,
        max_spatial
    ]
)

spatial_attention = layers.Conv2D(
    filters=1,
    kernel_size=7,
    padding="same",
    activation="sigmoid"
)(spatial_features)

output = layers.Multiply()(
    [
        refined_feature,
        spatial_attention
    ]
)
```

```

    return output

# -----
# PREPROCESSING FUNCTIONS
# -----

def preprocess_input(model_name, images):

    if model_name == "ResNet50":

        return resnet50.preprocess_input(images)

    elif model_name == "MobileNetV2":

        return mobilenet_v2.preprocess_input(images)

    elif model_name == "Xception":

        return xception.preprocess_input(images)

    elif model_name == "DenseNet121":

        return densenet.preprocess_input(images)

    # EfficientNet and ConvNeXt include
    # preprocessing internally.
    else:

        return images

# -----
# MODEL CREATION
# -----

def build_model(model_name):

    inputs = keras.Input(
        shape=(
            IMG_SIZE[0],
            IMG_SIZE[1],
            3
        ),
        name="input_image"
    )

    processed_inputs = layers.Lambda(
        lambda x: preprocess_input(
            model_name,
            x

```

```
)
)(inputs)

if model_name == "ResNet50":

    backbone = ResNet50(
        include_top=False,
        weights="imagenet",
        input_shape=(
            IMG_SIZE[0],
            IMG_SIZE[1],
            3
        )
    )

    use_cbam = False

elif model_name == "ResNet50_CBAM":

    backbone = ResNet50(
        include_top=False,
        weights="imagenet",
        input_shape=(
            IMG_SIZE[0],
            IMG_SIZE[1],
            3
        )
    )

    use_cbam = True

elif model_name == "EfficientNetB4":

    backbone = EfficientNetB4(
        include_top=False,
        weights="imagenet",
        input_shape=(
            IMG_SIZE[0],
            IMG_SIZE[1],
            3
        )
    )

    use_cbam = False

elif model_name == "ConvNeXtTiny":

    backbone = ConvNeXtTiny(
        include_top=False,
        weights="imagenet",
        input_shape=(
```

```
        IMG_SIZE[0],
        IMG_SIZE[1],
        3
    )
)

use_cbam = False

elif model_name == "MobileNetV2":

    backbone = MobileNetV2(
        include_top=False,
        weights="imagenet",
        input_shape=(
            IMG_SIZE[0],
            IMG_SIZE[1],
            3
        )
    )

    use_cbam = False

elif model_name == "Xception":

    backbone = Xception(
        include_top=False,
        weights="imagenet",
        input_shape=(
            IMG_SIZE[0],
            IMG_SIZE[1],
            3
        )
    )

    use_cbam = False

elif model_name == "DenseNet121":

    backbone = DenseNet121(
        include_top=False,
        weights="imagenet",
        input_shape=(
            IMG_SIZE[0],
            IMG_SIZE[1],
            3
        )
    )

    use_cbam = False

else:
```

```

        raise ValueError(
            "Unknown model: " + model_name
        )

    backbone.trainable = False

    features = backbone(
        processed_inputs,
        training=False
    )

    if use_cbam:

        features = cbam_block(
            features
        )

    features = layers.GlobalAveragePooling2D()(
        features
    )

    features = layers.BatchNormalization()(
        features
    )

    features = layers.Dropout(
        0.4
    )(features)

    outputs = layers.Dense(
        NUM_CLASSES,
        activation="softmax"
    )(features)

    model = keras.Model(
        inputs=inputs,
        outputs=outputs,
        name=model_name
    )

    return model

# -----
# CREATE ALL SEVEN MODELS
# -----

MODEL_NAMES = [
    "ResNet50",
    "ResNet50_CBAM",

```

```

    "EfficientNetB4",
    "ConvNeXtTiny",
    "MobileNetV2",
    "Xception",
    "DenseNet121"
]

models = {}

for model_name in MODEL_NAMES:

    print("\nCreating:", model_name)

    models[model_name] = build_model(
        model_name
    )

    print(
        "Parameters:",
        models[model_name].count_params()
    )

print("\nAll seven models created.")

print("MODULE 4 COMPLETED")

```

```

Creating: ResNet50
Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/94765736/94765736  1s 0us/step
Parameters: 23800804

Creating: ResNet50_CBAM
Parameters: 24851783

Creating: EfficientNetB4
Downloading data from https://storage.googleapis.com/keras-applications/efficientnet/71686520/71686520  1s 0us/step
Parameters: 17860291

Creating: ConvNeXtTiny
Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/111650432/111650432  1s 0us/step
Parameters: 27900100

Creating: MobileNetV2
Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/9406464/9406464  0s 0us/step
Parameters: 2391204

Creating: Xception
Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/83683744/83683744  1s 0us/step

```

Parameters: 21074572

Creating: DenseNet121

Downloading data from <https://storage.googleapis.com/tensorflow/keras-applications/29084464/29084464> 0s 0us/step

Parameters: 7144100

All seven models created.

MODULE 4 COMPLETED

```
# =====
# MODULE 5: TRAINING AND EVALUATION
# =====
```

```
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    classification_report,
    confusion_matrix,
    roc_auc_score
)

def compile_model(model):

    model.compile(

        optimizer=keras.optimizers.Adam(
            learning_rate=0.0003
        ),

        loss=keras.losses.CategoricalCrossentropy(
            label_smoothing=0.1
        ),

        metrics=[
            "accuracy"
        ]
    )

    return model

def evaluate_model(model, dataset):

    y_true = []
    y_probability = []

    for images, labels in dataset:
```

```
        predictions = model.predict(
            images,
            verbose=0
        )

        y_probability.extend(
            predictions
        )

        y_true.extend(
            labels.numpy()
        )

y_true = np.array(y_true)

y_probability = np.array(
    y_probability
)

true_classes = np.argmax(
    y_true,
    axis=1
)

predicted_classes = np.argmax(
    y_probability,
    axis=1
)

accuracy = accuracy_score(
    true_classes,
    predicted_classes
)

precision = precision_score(
    true_classes,
    predicted_classes,
    average="weighted",
    zero_division=0
)

recall = recall_score(
    true_classes,
    predicted_classes,
    average="weighted",
    zero_division=0
)

f1 = f1_score(
    true_classes,
```

```

        predicted_classes,
        average="weighted",
        zero_division=0
    )

    try:

        roc_auc = roc_auc_score(
            y_true,
            y_probability,
            multi_class="ovr",
            average="weighted"
        )

    except Exception:

        roc_auc = np.nan

    return {
        "accuracy": accuracy,
        "precision": precision,
        "recall": recall,
        "f1_score": f1,
        "roc_auc": roc_auc,
        "true_classes": true_classes,
        "predicted_classes": predicted_classes,
        "probabilities": y_probability
    }

# -----
# TRAIN ALL MODELS
# -----

histories = {}

evaluation_results = {}

for model_name in MODEL_NAMES:

    print("\n" + "=" * 60)

    print(
        "Training model:",
        model_name
    )

    print("=" * 60)

    model = models[model_name]

```

```
compile_model(model)

callbacks = [

    keras.callbacks.EarlyStopping(
        monitor="val_loss",
        patience=2,
        restore_best_weights=True
    ),

    keras.callbacks.ReduceLROnPlateau(
        monitor="val_loss",
        factor=0.3,
        patience=1,
        min_lr=1e-7
    )

]

start_time = time.time()

history = model.fit(

    train_dataset,

    validation_data=validation_dataset,

    epochs=EPOCHS,

    callbacks=callbacks
)

training_time = (
    time.time() - start_time
)

histories[model_name] = history

evaluation = evaluate_model(
    model,
    test_dataset
)

evaluation["parameters"] = (
    model.count_params()
)

evaluation["training_time_seconds"] = (
    training_time
)
```

```
evaluation_results[model_name] = evaluation

print("\nEvaluation results:")

print(
    "Accuracy:",
    evaluation["accuracy"]
)

print(
    "Precision:",
    evaluation["precision"]
)

print(
    "Recall:",
    evaluation["recall"]
)

print(
    "F1 Score:",
    evaluation["f1_score"]
)

print(
    "ROC-AUC:",
    evaluation["roc_auc"]
)

# Save model
model.save(
    OUTPUT_DIR / f"{model_name}.keras"
)

# -----
# COMPARISON TABLE
# -----

comparison_rows = []

for model_name, result in evaluation_results.items():

    comparison_rows.append({

        "Model": model_name,

        "Accuracy": result["accuracy"],

        "Precision": result["precision"],

        "Recall": result["recall"],
```

```

        "F1 Score": result["f1_score"],

        "ROC-AUC": result["roc_auc"],

        "Parameters": result["parameters"],

        "Training Time (seconds)": result[
            "training_time_seconds"
        ]

    })

comparison_df = pd.DataFrame(
    comparison_rows
)

comparison_df = comparison_df.sort_values(
    by="F1 Score",
    ascending=False
)

display(comparison_df)

comparison_df.to_csv(
    OUTPUT_DIR / "model_comparison.csv",
    index=False
)

# -----
# CONFUSION MATRICES
# -----

for model_name, result in evaluation_results.items():

    cm = confusion_matrix(
        result["true_classes"],
        result["predicted_classes"]
    )

    plt.figure(figsize=(14, 12))

    sns.heatmap(
        cm,
        cmap="Blues",
        xticklabels=class_names,
        yticklabels=class_names
    )

    plt.title(
        f"Confusion Matrix - {model_name}"
    )

```

```
)

plt.xlabel("Predicted Class")

plt.ylabel("Actual Class")

plt.xticks(rotation=90)

plt.yticks(rotation=0)

plt.tight_layout()

plt.show()

print(
    classification_report(
        result["true_classes"],
        result["predicted_classes"],
        target_names=class_names,
        zero_division=0
    )
)

print("MODULE 5 COMPLETED")
```


Evaluation results:

Accuracy: 0.7897884084636615

Precision: 0.807813151462547

Recall: 0.7897884084636615

F1 Score: 0.7881600705865055

ROC-AUC: 0.9956687934946787

=====
Training model: ResNet50_CBAM
=====

1268/1268  **169s** 122ms/step - accuracy: 0.1545 - loss: 4.02

Evaluation results:

Accuracy: 0.5726770929162833

Precision: 0.6243060029755482

Recall: 0.5726770929162833

F1 Score: 0.5616810757725227

ROC-AUC: 0.9781876147365101

=====
Training model: EfficientNetB4
=====

1268/1268  **226s** 136ms/step - accuracy: 0.4713 - loss: 2.76

Evaluation results:

Accuracy: 0.8376264949402024

Precision: 0.8491923609843989

Recall: 0.8376264949402024

F1 Score: 0.8344698830914788

ROC-AUC: 0.9977703354325207

=====
Training model: ConvNeXtTiny
=====

1268/1268  **179s** 129ms/step - accuracy: 0.3865 - loss: 3.09

Evaluation results:

Accuracy: 0.7833486660533578

Precision: 0.7998457691832634

Recall: 0.7833486660533578

F1 Score: 0.7800675517548027

ROC-AUC: 0.9967162293876827

=====
Training model: MobileNetV2
=====

1268/1268  **150s** 104ms/step - accuracy: 0.3094 - loss: 3.40

Evaluation results:

Accuracy: 0.7060717571297148

Precision: 0.7344317277496055
Recall: 0.7060717571297148
F1 Score: 0.698773100924623
ROC-AUC: 0.9923379423979648

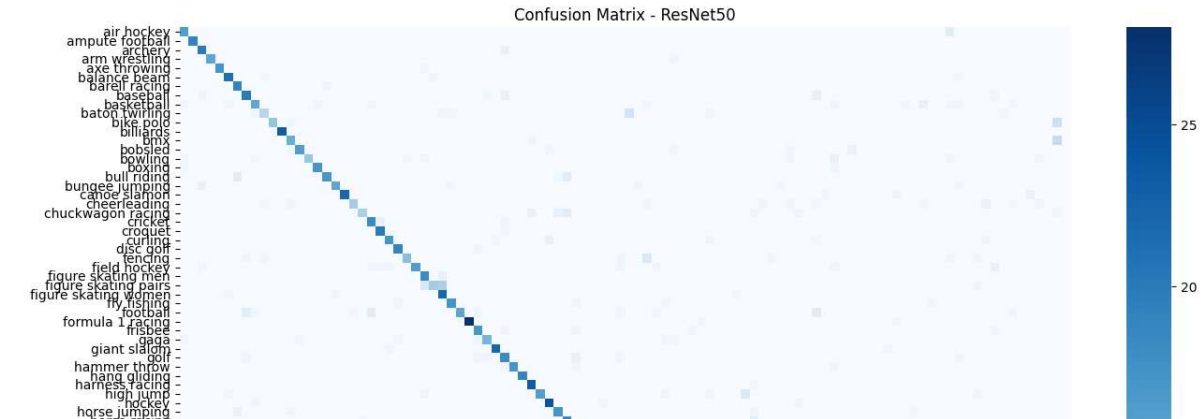
=====
Training model: Xception
=====
1268/1268 **196s** 138ms/step - accuracy: 0.3976 - loss: 3.04

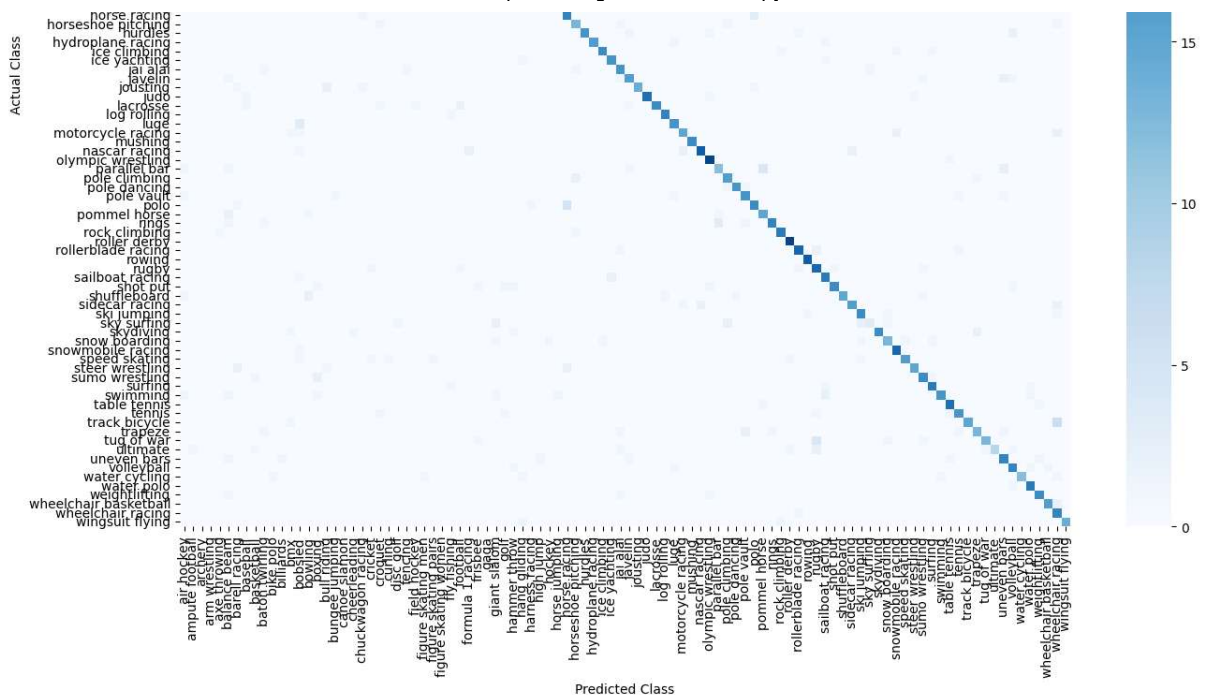
Evaluation results:
Accuracy: 0.7437902483900644
Precision: 0.7626917002273198
Recall: 0.7437902483900644
F1 Score: 0.7385074904241429
ROC-AUC: 0.994585102475071

=====
Training model: DenseNet121
=====
1268/1268 **196s** 133ms/step - accuracy: 0.2820 - loss: 3.43

Evaluation results:
Accuracy: 0.7281508739650414
Precision: 0.749947214922406
Recall: 0.7281508739650414
F1 Score: 0.7186593036379901
ROC-AUC: 0.9929602717562417

	Model	Accuracy	Precision	Recall	F1 Score	ROC-AUC	Parameters
2	EfficientNetB4	0.837626	0.849192	0.837626	0.834470	0.997770	1786029
0	ResNet50	0.789788	0.807813	0.789788	0.788160	0.995669	2380080
3	ConvNeXtTiny	0.783349	0.799846	0.783349	0.780068	0.996716	2790010
5	Xception	0.743790	0.762692	0.743790	0.738507	0.994585	2107457
6	DenseNet121	0.728151	0.749947	0.728151	0.718659	0.992960	714410
4	MobileNetV2	0.706072	0.734432	0.706072	0.698773	0.992338	239120
1	ResNet50_CBAM	0.572677	0.624306	0.572677	0.561681	0.978188	2485178

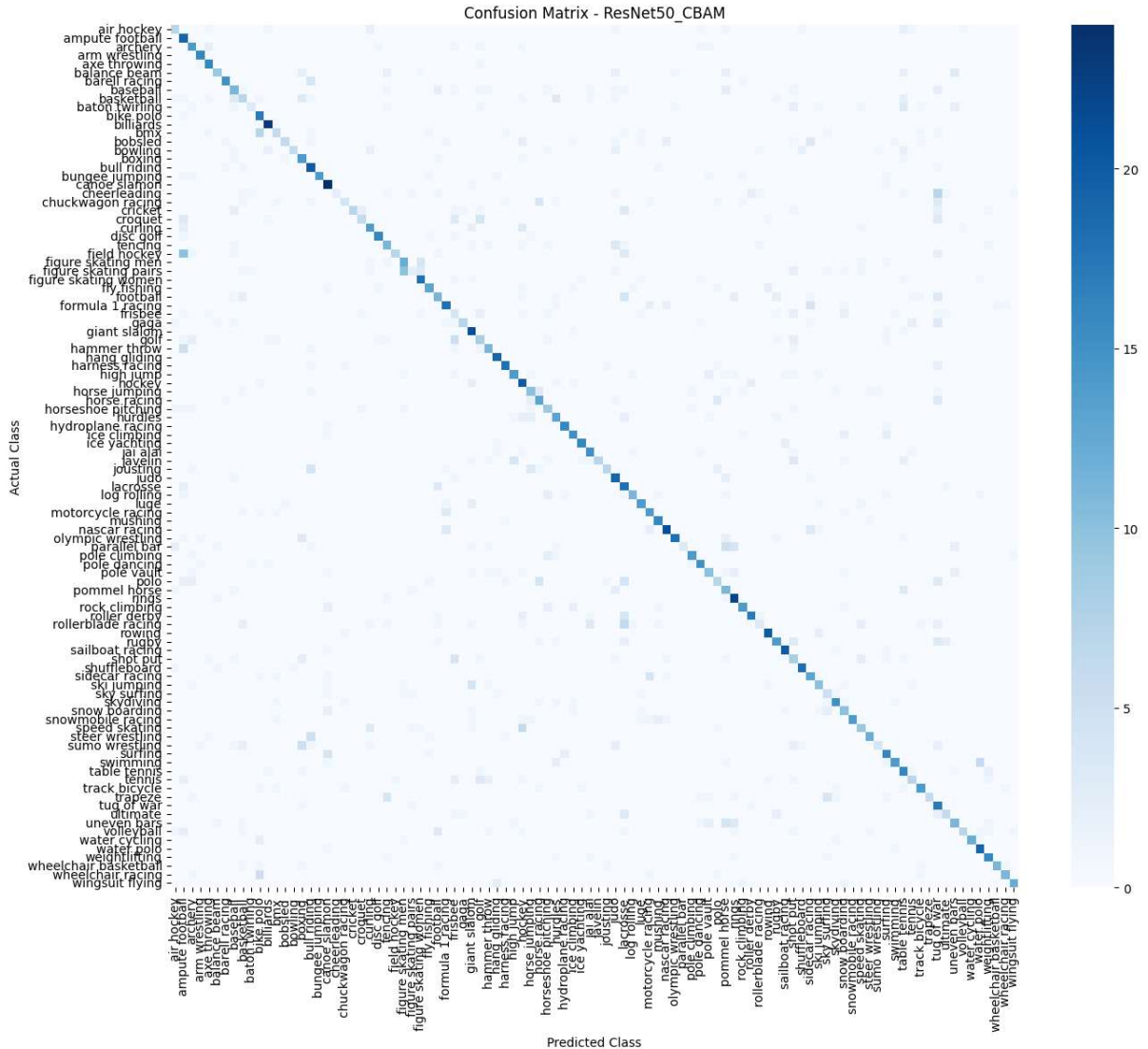




	precision	recall	f1-score	support
air hockey	0.67	0.84	0.74	19
amputee football	0.95	1.00	0.97	19
archery	0.83	0.91	0.87	22
arm wrestling	1.00	0.94	0.97	16
axe throwing	0.94	0.94	0.94	18
balance beam	0.66	0.88	0.75	24
barell racing	0.68	0.95	0.79	20
baseball	0.74	0.74	0.74	27
basketball	0.71	0.56	0.62	27
baton twirling	0.57	0.44	0.50	18
bike polo	0.92	0.61	0.73	18
billiards	0.96	1.00	0.98	23
bmj	0.67	0.64	0.65	22
bobsled	0.64	0.73	0.68	22
bowling	0.79	0.55	0.65	20
boxing	0.77	0.89	0.83	19
bull riding	0.81	0.71	0.76	24
bungee jumping	0.88	0.71	0.79	21
canoe slalom	1.00	0.85	0.92	26
cheerleading	0.83	0.48	0.61	21
chuckwagon racing	0.82	0.47	0.60	19
cricket	0.72	0.86	0.78	21
croquet	0.77	0.95	0.85	21
curling	0.85	0.74	0.79	23
disc golf	0.86	0.95	0.90	20
fencing	0.86	0.55	0.67	22
field hockey	0.94	0.64	0.76	25
figure skating men	0.69	0.90	0.78	20
figure skating pairs	0.83	0.42	0.56	24
figure skating women	0.65	0.88	0.75	25
fly fishing	0.81	0.81	0.81	21
football	0.83	0.50	0.62	30
formula 1 racing	0.93	0.93	0.93	30
frisbee	0.74	0.81	0.77	21
giant slalom	0.87	0.68	0.76	19

	0.87	0.88	0.79	27
giant slalom	0.81	0.92	0.86	24
golf	0.64	0.72	0.68	25
hammer throw	0.85	0.85	0.85	20
hang gliding	0.86	1.00	0.93	19
harness racing	0.85	0.96	0.90	24
high jump	0.94	0.64	0.76	25
hockey	0.86	0.89	0.87	27
horse jumping	0.81	0.77	0.79	22
horse racing	0.59	0.83	0.69	23
horseshoe pitching	0.59	0.76	0.67	17
hurdles	1.00	0.77	0.87	22
hydroplane racing	1.00	0.89	0.94	18
ice climbing	0.90	0.86	0.88	21
ice yachting	0.89	0.89	0.89	19
jai alai	0.63	0.81	0.71	21
javelin	0.64	0.73	0.68	22
jousting	0.93	0.74	0.82	19
judo	0.78	0.88	0.82	24
lacrosse	0.90	0.75	0.82	24
log rolling	0.83	0.95	0.88	20
luge	0.74	0.85	0.79	20
motorcycle racing	0.83	0.71	0.77	21
mushing	0.95	1.00	0.97	18
nascar racing	0.88	0.77	0.82	30
olympic wrestling	0.79	0.96	0.87	27
parallel bar	0.75	0.57	0.65	21
pole climbing	0.80	0.84	0.82	19
pole dancing	0.89	0.89	0.89	19
pole vault	0.68	0.77	0.72	22
polo	0.72	0.72	0.72	25
pommel horse	0.71	0.79	0.75	19
rings	0.90	0.76	0.83	25
rock climbing	0.87	0.91	0.89	22
roller derby	0.84	1.00	0.91	26
rollerblade racing	0.81	0.85	0.83	26
rowing	0.96	1.00	0.98	23
rugby	0.58	0.81	0.68	27
sailboat racing	0.77	0.87	0.82	23
shot put	0.72	0.75	0.73	24
shuffleboard	1.00	0.71	0.83	21
sidecar racing	0.80	0.76	0.78	21
ski jumping	0.82	0.86	0.84	21
sky surfing	0.75	0.27	0.40	11
skydiving	1.00	0.78	0.88	23
snow boarding	0.76	0.68	0.72	19
snowmobile racing	0.81	0.96	0.88	23
speed skating	0.89	0.73	0.80	22
steer wrestling	0.88	0.75	0.81	20
sumo wrestling	0.72	0.82	0.77	22
surfing	0.83	0.87	0.85	23
swimming	0.94	0.68	0.79	25
table tennis	0.70	0.91	0.79	23
tennis	0.71	0.81	0.76	21
track bicycle	1.00	0.68	0.81	22
trapeze	0.68	0.65	0.67	20
tug of war	0.76	0.65	0.70	20

ultimate	0.67	0.50	0.57	16
uneven bars	0.73	0.76	0.75	25
volleyball	0.68	0.83	0.75	23
water cycling	1.00	0.71	0.83	17
water polo	0.80	0.95	0.87	21
weightlifting	0.82	0.86	0.84	21
wheelchair basketball	0.94	0.89	0.91	18
wheelchair racing	0.40	0.95	0.56	20
wingsuit flying	1.00	0.88	0.93	16
accuracy			0.79	2174
macro avg	0.81	0.79	0.79	2174
weighted avg	0.81	0.79	0.79	2174

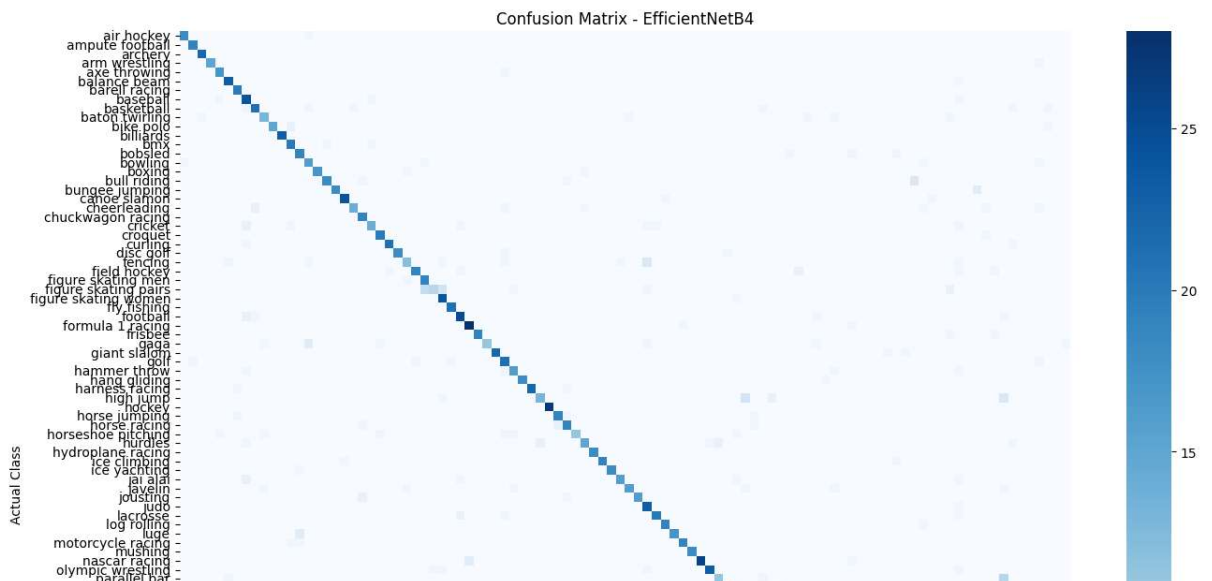


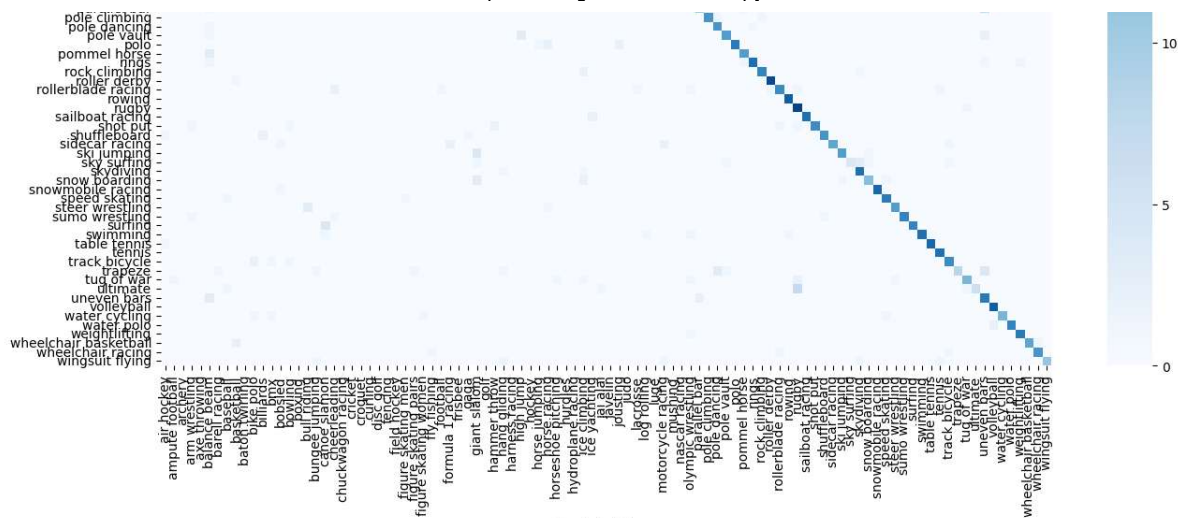
	precision	recall	f1-score	support
--	-----------	--------	----------	---------

air hockey	0.47	0.37	0.41	19
amputee football	0.34	1.00	0.51	19
archery	0.50	0.64	0.56	22
arm wrestling	0.80	1.00	0.89	16
axe throwing	0.67	0.89	0.76	18
balance beam	0.64	0.38	0.47	24
barell racing	0.83	0.75	0.79	20
baseball	0.41	0.41	0.41	27
baskethall	0.30	0.26	0.28	27

baton twirling	0.50	0.17	0.25	18
bike polo	0.46	0.94	0.62	18
billiards	0.77	1.00	0.87	23
bmx	0.46	0.27	0.34	22
bobsled	0.75	0.27	0.40	22
bowling	0.86	0.30	0.44	20
boxing	0.44	0.74	0.55	19
bull riding	0.48	0.83	0.61	24
bungee jumping	1.00	0.67	0.80	21
canoe slamon	0.57	0.92	0.71	26
cheerleading	1.00	0.10	0.17	21
chuckwagon racing	0.50	0.21	0.30	19
cricket	1.00	0.33	0.50	21
croquet	0.60	0.29	0.39	21
curling	0.56	0.61	0.58	23
disc golf	0.84	0.80	0.82	20
fencing	0.44	0.50	0.47	22
field hockey	0.88	0.28	0.42	25
figure skating men	0.44	0.60	0.51	20
figure skating pairs	0.29	0.08	0.13	24
figure skating women	0.64	0.72	0.68	25
fly fishing	0.68	0.62	0.65	21
football	0.42	0.37	0.39	30
formula 1 racing	0.64	0.60	0.62	30
frisbee	0.14	0.19	0.16	21
gaga	0.78	0.37	0.50	19
giant slalom	0.51	0.88	0.65	24
golf	0.32	0.32	0.32	25
hammer throw	0.55	0.55	0.55	20
hang gliding	0.73	1.00	0.84	19
harness racing	0.78	0.75	0.77	24
high jump	0.70	0.56	0.62	25
hockey	0.42	0.74	0.53	27
horse jumping	0.48	0.45	0.47	22
horse racing	0.46	0.57	0.51	23
horseshoe pitching	0.43	0.53	0.47	17
hurdles	0.48	0.59	0.53	22
hydroplane racing	0.84	0.89	0.86	18
ice climbing	0.65	0.71	0.68	21
ice yachting	0.84	0.84	0.84	19
jai alai	0.62	0.71	0.67	21
javelin	1.00	0.32	0.48	22
jousting	0.70	0.37	0.48	19
judo	0.50	0.79	0.61	24
lacrosse	0.33	0.75	0.46	24
log rolling	0.73	0.55	0.63	20
luge	0.67	0.70	0.68	20
motorcycle racing	0.56	0.67	0.61	21
mushing	0.73	0.89	0.80	18
nascar racing	0.75	0.70	0.72	30
olympic wrestling	0.90	0.67	0.77	27
parallel bar	1.00	0.14	0.25	21
pole climbing	0.82	0.74	0.78	19
pole dancing	0.68	0.79	0.73	19
pole vault	0.56	0.45	0.50	22
polo	0.41	0.28	0.33	25

pommel horse	0.31	0.58	0.40	19
rings	0.59	0.88	0.71	25
rock climbing	0.70	0.64	0.67	22
roller derby	0.77	0.65	0.71	26
rollerblade racing	0.43	0.12	0.18	26
rowing	0.80	0.87	0.83	23
rugby	0.61	0.52	0.56	27
sailboat racing	0.83	0.87	0.85	23
shot put	0.24	0.33	0.28	24
shuffleboard	0.82	0.86	0.84	21
sidecar racing	0.50	0.62	0.55	21
ski jumping	0.59	0.48	0.53	21
sky surfing	0.33	0.45	0.38	11
skydiving	0.71	0.65	0.68	23
snow boarding	0.43	0.53	0.48	19
snowmobile racing	1.00	0.61	0.76	23
speed skating	0.50	0.41	0.45	22
steer wrestling	0.80	0.60	0.69	20
sumo wrestling	0.67	0.18	0.29	22
surfing	0.55	0.70	0.62	23
swimming	0.78	0.56	0.65	25
table tennis	0.41	0.70	0.52	23
tennis	0.50	0.33	0.40	21
track bicycle	0.67	0.64	0.65	22
trapeze	1.00	0.30	0.46	20
tug of war	0.27	0.85	0.41	20
ultimate	0.55	0.38	0.44	16
uneven bars	0.52	0.44	0.48	25
volleyball	0.78	0.30	0.44	23
water cycling	1.00	0.71	0.83	17
water polo	0.66	0.90	0.76	21
weightlifting	0.70	0.76	0.73	21
wheelchair basketball	0.92	0.61	0.73	18
wheelchair racing	0.79	0.55	0.65	20
wingsuit flying	0.80	0.75	0.77	16
accuracy			0.57	2174
macro avg	0.63	0.58	0.56	2174
weighted avg	0.62	0.57	0.56	2174

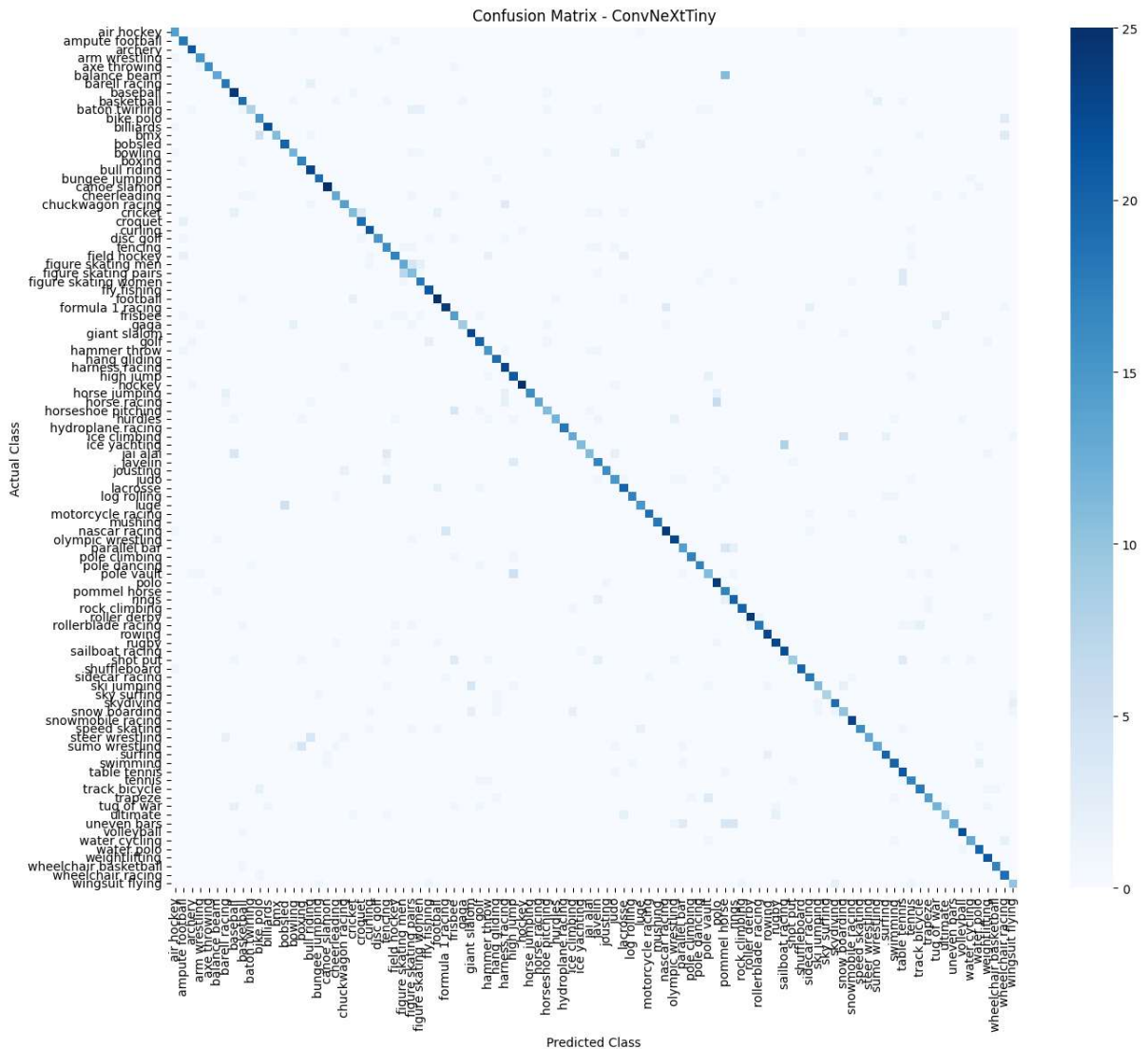




	precision	recall	f1-score	support
air hockey	0.86	0.95	0.90	19
amputee football	0.90	1.00	0.95	19
archery	0.96	1.00	0.98	22
arm wrestling	0.88	0.94	0.91	16
axe throwing	0.89	0.94	0.92	18
balance beam	0.68	0.96	0.79	24
barell racing	0.87	1.00	0.93	20
baseball	0.67	0.89	0.76	27
basketball	0.78	0.78	0.78	27
baton twirling	0.81	0.72	0.76	18
bike polo	0.83	0.83	0.83	18
billiards	0.92	1.00	0.96	23
bmj	0.77	0.91	0.83	22
bobsled	0.73	0.86	0.79	22
bowling	0.67	0.80	0.73	20
boxing	1.00	0.89	0.94	19
bull riding	0.82	0.75	0.78	24
bungee jumping	0.90	0.86	0.88	21
canoe slamon	0.80	0.92	0.86	26
cheerleading	0.78	0.67	0.72	21
chuckwagon racing	0.83	1.00	0.90	19
cricket	0.88	0.67	0.76	21
croquet	0.91	0.95	0.93	21
curling	0.95	0.91	0.93	23
disc golf	1.00	0.90	0.95	20
fencing	0.80	0.55	0.65	22
field hockey	0.95	0.76	0.84	25
figure skating men	0.68	0.95	0.79	20
figure skating pairs	0.80	0.33	0.47	24
figure skating women	0.73	0.96	0.83	25
fly fishing	0.91	1.00	0.95	21
football	0.81	0.83	0.82	30
formula 1 racing	0.85	0.93	0.89	30
frisbee	1.00	0.90	0.95	21
gaga	0.92	0.58	0.71	19
giant slalom	0.73	0.92	0.81	24
golf	0.75	0.84	0.79	25
hammer throw	0.76	0.80	0.78	20
hang gliding	0.86	0.95	0.90	19

harness racing	1.00	0.92	0.96	24
high jump	0.72	0.52	0.60	25
hockey	1.00	1.00	1.00	27
horse jumping	0.86	0.86	0.86	22
horse racing	0.73	0.83	0.78	23
horseshoe pitching	0.92	0.65	0.76	17
hurdles	0.88	0.68	0.77	22
hydroplane racing	1.00	1.00	1.00	18
ice climbing	0.76	0.90	0.83	21
ice yachting	0.90	0.95	0.92	19
jai alai	0.89	0.76	0.82	21
javelin	0.94	0.73	0.82	22
jousting	0.89	0.84	0.86	19
judo	0.74	0.96	0.84	24
lacrosse	0.91	0.83	0.87	24
log rolling	0.90	0.95	0.93	20
luge	1.00	0.85	0.92	20
motorcycle racing	0.83	0.90	0.86	21
mushing	1.00	1.00	1.00	18
nascar racing	1.00	0.87	0.93	30
olympic wrestling	0.82	0.85	0.84	27
parallel bar	0.73	0.52	0.61	21
pole climbing	0.94	0.89	0.92	19
pole dancing	0.71	0.89	0.79	19
pole vault	0.67	0.73	0.70	22
polo	0.91	0.80	0.85	25
pommel horse	0.84	0.84	0.84	19
rings	0.88	0.84	0.86	25
rock climbing	0.95	0.86	0.90	22
roller derby	0.93	0.96	0.94	26
rollerblade racing	0.82	0.69	0.75	26
rowing	0.92	1.00	0.96	23
rugby	0.67	0.96	0.79	27
sailboat racing	0.95	0.91	0.93	23
shot put	0.86	0.75	0.80	24
shuffleboard	0.94	0.81	0.87	21
sidecar racing	0.83	0.71	0.77	21
ski jumping	0.94	0.76	0.84	21
sky surfing	1.00	0.36	0.53	11
skydiving	0.81	0.91	0.86	23
snow boarding	0.80	0.63	0.71	19
snowmobile racing	0.92	0.96	0.94	23
speed skating	0.87	0.91	0.89	22
steer wrestling	0.76	0.80	0.78	20
sumo wrestling	0.86	0.86	0.86	22
surfing	0.95	0.83	0.88	23
swimming	1.00	0.84	0.91	25
table tennis	0.85	0.96	0.90	23
tennis	0.64	1.00	0.78	21
track bicycle	0.86	0.82	0.84	22
trapeze	0.73	0.40	0.52	20
tug of war	0.76	0.65	0.70	20
ultimate	0.75	0.38	0.50	16
uneven bars	0.49	0.80	0.61	25
volleyball	0.82	1.00	0.90	23
water cycling	1.00	0.76	0.87	17
water polo	1.00	0.90	0.95	21

weightlifting	0.80	0.95	0.87	21
wheelchair basketball	0.88	0.83	0.86	18
wheelchair racing	0.94	0.85	0.89	20
wingsuit flying	0.92	0.69	0.79	16
accuracy			0.84	2174
macro avg	0.85	0.83	0.83	2174
weighted avg	0.85	0.84	0.83	2174

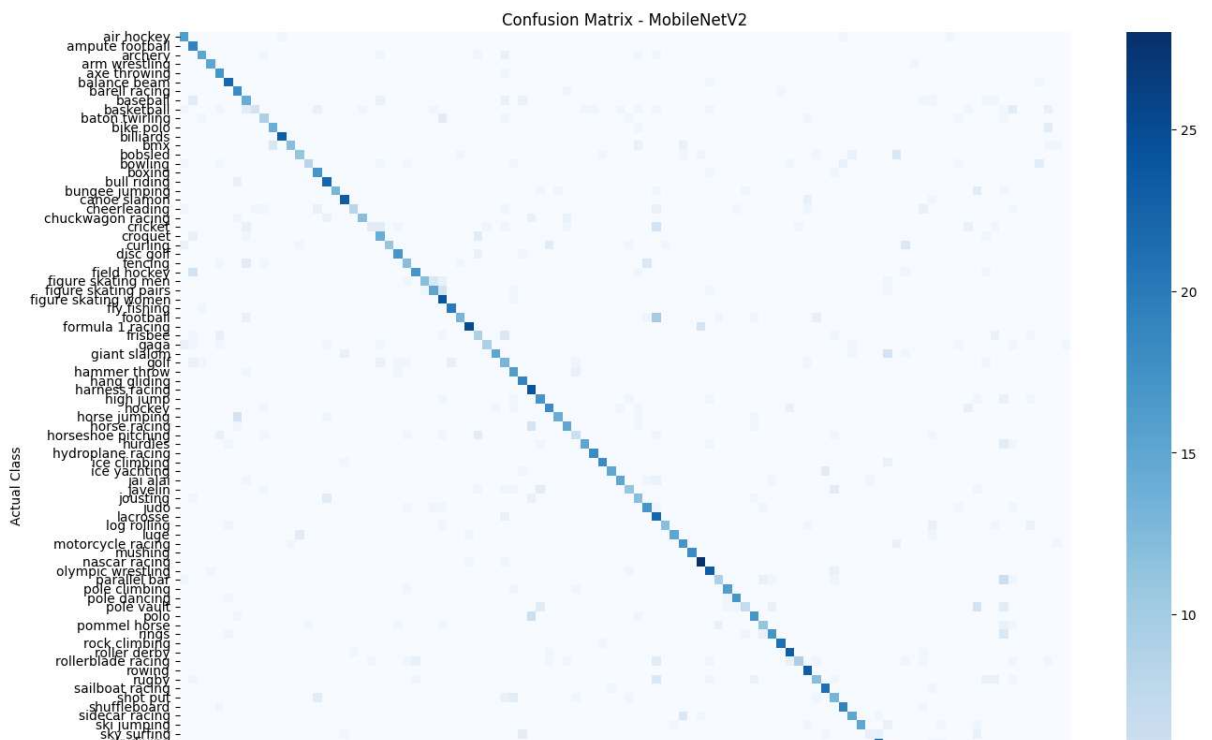


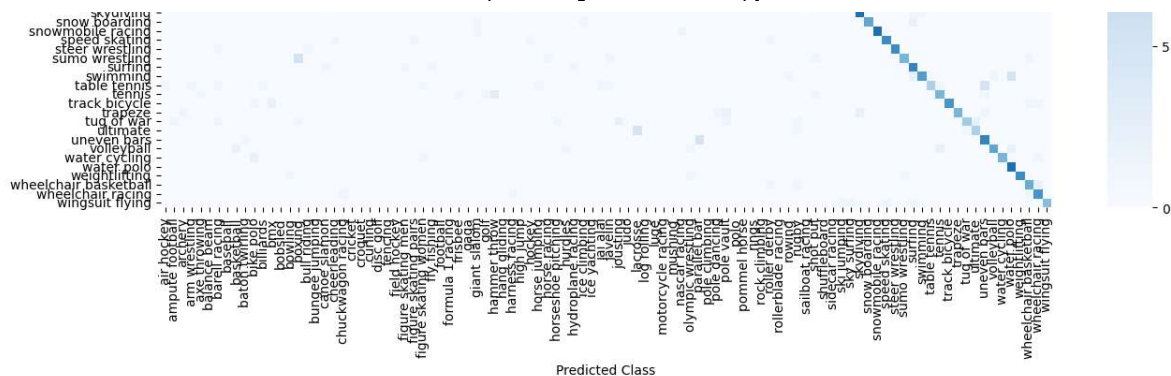
	precision	recall	f1-score	support
--	-----------	--------	----------	---------

air hockey	0.74	0.74	0.74	19
amputee football	0.72	0.95	0.82	19
archery	0.84	0.95	0.89	22
arm wrestling	0.83	0.94	0.88	16
axe throwing	1.00	0.89	0.94	18
balance beam	0.87	0.54	0.67	24
barell racing	0.78	0.90	0.84	20
baseball	0.69	0.89	0.77	27
basketball	0.73	0.70	0.72	27
baton twirling	0.80	0.44	0.57	18
bike polo	0.65	0.83	0.73	18
billiards	0.96	0.96	0.96	23
bmX	1.00	0.50	0.67	22

bobsled	0.80	0.91	0.85	22
bowling	0.71	0.60	0.65	20
boxing	0.74	0.89	0.81	19
bull riding	0.72	0.96	0.82	24
bungee jumping	0.90	0.90	0.90	21
canoe slalom	0.93	0.96	0.94	26
cheerleading	0.76	0.62	0.68	21
chuckwagon racing	0.74	0.74	0.74	19
cricket	0.73	0.52	0.61	21
croquet	0.83	0.90	0.86	21
curling	0.84	0.91	0.88	23
disc golf	0.88	0.75	0.81	20
fencing	0.59	0.73	0.65	22
field hockey	0.77	0.68	0.72	25
figure skating men	0.57	0.65	0.60	20
figure skating pairs	0.46	0.46	0.46	24
figure skating women	0.72	0.72	0.72	25
fly fishing	0.84	1.00	0.91	21
football	0.81	0.83	0.82	30
formula 1 racing	0.83	0.80	0.81	30
frisbee	0.54	0.67	0.60	21
gaga	0.90	0.47	0.62	19
giant slalom	0.79	0.96	0.87	24
golf	0.77	0.80	0.78	25
hammer throw	0.68	0.75	0.71	20
hang gliding	0.79	1.00	0.88	19
harness racing	0.72	0.96	0.82	24
high jump	0.66	0.84	0.74	25
hockey	0.96	0.93	0.94	27
horse jumping	1.00	0.73	0.84	22
horse racing	1.00	0.57	0.72	23
horseshoe pitching	0.69	0.65	0.67	17
hurdles	0.92	0.55	0.69	22
hydroplane racing	1.00	1.00	1.00	18
ice climbing	0.81	0.62	0.70	21
ice yachting	0.92	0.58	0.71	19
jai alai	0.73	0.52	0.61	21
javelin	0.61	0.77	0.68	22
jousting	0.94	0.84	0.89	19
judo	0.65	0.62	0.64	24
lacrosse	0.74	0.83	0.78	24
log rolling	0.94	0.85	0.89	20
luge	0.75	0.75	0.75	20
motorcycle racing	0.90	0.90	0.90	21
mushing	1.00	1.00	1.00	18
nascar racing	0.86	0.80	0.83	30
olympic wrestling	0.82	0.85	0.84	27
parallel bar	0.67	0.67	0.67	21
pole climbing	0.94	0.89	0.92	19
pole dancing	0.89	0.89	0.89	19
pole vault	0.55	0.50	0.52	22
polo	0.69	0.96	0.80	25
pommel horse	0.42	0.89	0.58	19
rings	0.69	0.80	0.74	25
rock climbing	0.95	0.91	0.93	22
roller derby	0.96	0.92	0.94	26
rollerblade racing	0.90	0.69	0.78	26

rowing	0.88	1.00	0.94	23
rugby	0.85	0.85	0.85	27
sailboat racing	0.73	0.96	0.83	23
shot put	0.82	0.38	0.51	24
shuffleboard	0.87	0.95	0.91	21
sidecar racing	0.82	0.86	0.84	21
ski jumping	0.85	0.52	0.65	21
sky surfing	1.00	0.73	0.84	11
skydiving	0.86	0.83	0.84	23
snow boarding	0.56	0.53	0.54	19
snowmobile racing	0.88	1.00	0.94	23
speed skating	1.00	0.73	0.84	22
steer wrestling	0.87	0.65	0.74	20
sumo wrestling	0.76	0.59	0.67	22
surfing	0.87	0.87	0.87	23
swimming	0.83	0.80	0.82	25
table tennis	0.57	0.91	0.70	23
tennis	0.65	0.81	0.72	21
track bicycle	0.86	0.82	0.84	22
trapeze	0.82	0.70	0.76	20
tug of war	0.80	0.60	0.69	20
ultimate	0.71	0.62	0.67	16
uneven bars	0.93	0.52	0.67	25
volleyball	0.88	0.96	0.92	23
water cycling	0.87	0.76	0.81	17
water polo	0.83	0.95	0.89	21
weightlifting	0.81	1.00	0.89	21
wheelchair basketball	0.94	0.94	0.94	18
wheelchair racing	0.68	0.95	0.79	20
wingsuit flying	0.67	0.62	0.65	16
accuracy			0.78	2174
macro avg	0.80	0.78	0.78	2174
weighted avg	0.80	0.78	0.78	2174





	precision	recall	f1-score	support
air hockey	0.64	0.84	0.73	19
amputee football	0.53	1.00	0.69	19
archery	0.79	0.68	0.73	22
arm wrestling	0.88	0.94	0.91	16
axe throwing	0.63	0.94	0.76	18
balance beam	0.85	0.92	0.88	24
barell racing	0.58	0.90	0.71	20
baseball	0.52	0.52	0.52	27
basketball	0.62	0.19	0.29	27
baton twirling	0.56	0.50	0.53	18
bike polo	0.67	0.78	0.72	18
billiards	0.92	1.00	0.96	23
bmj	0.80	0.55	0.65	22
bobsled	0.73	0.50	0.59	22
bowling	0.80	0.40	0.53	20
boxing	0.57	0.89	0.69	19
bull riding	0.73	0.92	0.81	24
bungee jumping	1.00	0.62	0.76	21
canoe slalom	0.82	0.88	0.85	26
cheerleading	0.80	0.38	0.52	21
chuckwagon racing	0.86	0.63	0.73	19
cricket	0.75	0.14	0.24	21
croquet	0.58	0.67	0.62	21
curling	1.00	0.48	0.65	23
disc golf	0.81	0.85	0.83	20
fencing	0.63	0.55	0.59	22
field hockey	0.85	0.68	0.76	25
figure skating men	0.86	0.60	0.71	20
figure skating pairs	0.68	0.62	0.65	24
figure skating women	0.57	0.96	0.72	25
fly fishing	0.87	0.95	0.91	21
football	0.87	0.43	0.58	30
formula 1 racing	0.93	0.83	0.88	30
frisbee	0.45	0.43	0.44	21
gaga	0.90	0.47	0.62	19
giant slalom	0.83	0.62	0.71	24
golf	0.41	0.52	0.46	25
hammer throw	0.55	0.80	0.65	20
hang gliding	0.73	1.00	0.84	19
harness racing	0.60	1.00	0.75	24
high jump	0.68	0.68	0.68	25
hockey	0.75	0.67	0.71	27
horse jumping	0.88	0.64	0.74	22
horse racing	0.71	0.65	0.68	23